

HelixCluster

HelixCluster

데이터센터 **GPU**부터 엣지 핸드헬드까지 **AI** 컴퓨팅을 아우르는 분산 운영 체제 — 단일 제어 플레인 하에서.

요약

Helix 클러스터 OS는 차세대 분산 운영 체제로, 데이터센터 GPU부터 엣지 SBC, 핸드헬드에 이르기까지 이기종 노드 전반의 컴퓨팅을 오케스트레이션합니다. HPC 스케줄링, 컨테이너 오케스트레이션, AI/ML 추론, 페더레이티드 멀티 클러스터 운영, 보안 멀티테넌트 세션을 단일 제어 플레인 하에 통합합니다.

간략 설명

Go 기반 분산 OS / GPU 공유 컴퓨팅 클러스터. HPC 스케줄링(오메가 모델의 이중 레벨 스케줄러), 컨테이너 오케스트레이션, AI 추론 라우팅, 페더레이션, 보안 멀티테넌트 세션을 이기종 노드 전반에서 통합하며, SWIM 가십 프로토콜과 래프트 합의로 조정됩니다. 양자 내성 종단 간 암호화를 적용했습니다.

상세 설명

Helix 클러스터 OS는 데이터센터 GPU, 엣지 싱글보드 컴퓨터, 심지어 핸드헬드까지 극도로 이기종인 하드웨어 전반의 컴퓨팅 워크로드를 단일 제어 플레인 하에 오케스트레이션합니다. A100 랙과 소수의 SBC를 수십 개의 호환되지 않는 섬이 아닌 하나의 주소 지정 가능한 패브릭으로 취급합니다. 이는 L0 하드웨어 계층부터 L7 페더레이션 및 관측성 계층까지 7단계 스택을 구현한 Go 워크스페이스(모노레포 및 깃 서브모듈)로, 14개의 제어 플레인 마이크로서비스로 조정됩니다. 노드 멤버십은 SWIM 가십 프로토콜과 디스커버리로 추적되어 노드의 참여와 이탈에 따라 패브릭이 자가 치유되며, 강력한 일관성을 보장하는 상태는 래프트 합의를 통해 관리됩니다. 래프트 그룹은 샤드별로 구성되어 리스홀더 로컬 읽기로 속도를 높이고, STONITH 펜싱으로 파티션된 노드가 공유 상태를 손상시키지 않도록 보장합니다.

워크로드 배치는 오메가 모델의 이중 레벨 스케줄러를 통해 이루어집니다. 낙관적 동시성, ClassAd 매칭, 갭 스케줄링, 가치 기반 선점, 제약 조건 기반 배치를 지원하며, 기존 HPC 스케줄러의 한계를 넘어섭니다. 탄소 인식 및 비용/TCO 인식 라우팅, 클라우드로의 버스트 오토스케일링, 마켓플레이스 어댑터(Akash, io.net, RunPod, AWS Spot, Chutes)를 통해 로컬 자원이 부족할 경우 임대 용량으로 작업이 확장됩니다.

최종 사용자는 이러한 복잡한 메커니즘을 직접 접하지 않습니다. 이들은 깔끔한 세션 모델(컴퓨팅 할당), 대화형 WebSocket/PTY 터미널, 내부 AI 추론 라우트, 풀 활용도 조회를 통해 상호작용합니다. 보안은 부가 기능이 아닌 핵심 계층으로 설계되었습니다. SPIFFE 신원 인증, 디바이스 증명(챌린지/응답, GPU 작업 증명, 봉인), 수출 통제 KYC 게이트, 양자 내성 종단 간 암호화 전송 계층을 제공합니다. X25519 + ML-KEM-768 하이브리드 키 교환에 AEAD 레코드 보호 및 재생 공격 방지 기능을 더해, 오늘 캡처된 트래픽도 미래의 양자 공격으로부터 안전하게 보호됩니다.

정확성은 주장이 아닌 입증됩니다. 결정론적 시뮬레이션 테스트 (FoundationDB 스타일의 시드 실행, 장애 주입, 네트워크 시뮬레이션, 바이트 단위 재생, Porcupine 선형성 검사기)를 통해 분산 장애를 필요에 따라 재현하며, 필수 쌍돌연변이 테스트로 가드 테스트의 유효성을 입증합니다. 아키텍처와 문서는 기계적 린트로 지속적으로 검증되어 현실과 문서 간의 불일치가 발생하는 즉시 빌드를 실패시킵니다.

왜 만들었는가

AI와 HPC 워크로드를 전혀 다른 하드웨어 계층에서 실행하되, 별도의 스케줄러, 오케스트레이터, 추론 스택을 짜깁기하지 않고도 가능하게 하기 위해서다. 또한 모든 출시 기능이 실제 최종 사용자 행동을 입증해야 한다는 엔지니어링 보장을 제공하며(그린 테스트나 스텝을 절대 사용하지 않음), 모든 플랫폼별 기능은 해당 플랫폼의 실제 네이티브 기능을 활용하도록(Linux 전용 모의 환경을 사용하지 않음) 설계되었다. 저장소 거버넌스에 인용된 동기는 “테스트는 통과했지만 기능은 실제로 작동하지 않는다”는 실패 모드이며, 이 프로젝트는 이를 명시적으로 제거하기 위해 구축되었다.

왜 혁신적인가

일반적으로 다섯 가지 별도의 스택—HPC 스케줄링, 컨테이너 오케스트레이션, AI 추론, 멀티 클러스터 페더레이션, 보안 멀티테넌트 세션—을 하나로 통합한 단일 제어 플레인을 제공한다. 이 제어 플레인은 데이터센터 GPU부터 엣지 핸드헬드 기기까지 아우르며, 보통 전문 인프라에서나 볼 수 있는 엄격한 기준을 적용한다. 형식적 방법론 수준의 정확성(TLA+ 명세, 결정론적 시뮬레이션, 선형화 가능성 검증)과 양자 후 암호화 기밀 전송을 보장하며, 대부분의 오케스트레이터가 시도조차 하지 않는 수준이다. 기술적 차별화뿐만 아니라, 비용 및 탄소 인식 배치와 클라우드 마켓플레이스 버스트 기능을 통해 경제적 레버리지까지 제공한다. 스케줄러는 자동으로 더 저렴하거나 친환경적인 용량, 혹은 여유 용량을 찾아내므로, 동일한 워크로드를 더 적은 비용과 탄소 배출로 실행할 수 있으며, 사용자는 작업을 다시 작성할 필요가 없다.

무엇이 혁신적인가

- 결정론적 시뮬레이션 테스트(**DST**) — 시드 기반의 완벽히 재현 가능한 시뮬레이터로, 장애, 시계 왜곡, 네트워크 분할을 주입하고 이를 바이트 단위로 재생한 후, 포큐파인 선형화 가능성 검증기를 통해 실행한다. 이로써 한 번 발견된 하이젠버그는 언제든지 재현할 수 있게 된다.
- 오메가 모델 2단계 스케줄러 — 낙관적 동시성 배치와 ClassAd 매칭, 갭 스케줄링, 가치 기반 선점을 지원하는 공유 상태 설계로, 중앙 병목 현상 없이 여러 스케줄러가 하나의 클러스터에 커밋할 수 있다.
- 양자 후 **E2EE**/기밀 추론 연결 — X25519 + ML-KEM-768 하이브리드 키 교환 방식으로, 요청마다 응답 키페어 바인딩과 AEAD(재생 공격 방지)를 적용한다. 암호화 기본 요소는 실제 검증되었으며, 전체 기밀 멀티노드 라운드트립은 명시적으로 계획/게이트 단계에 있다.
- 증명 기반 신뢰 — 노드는 자신이 무엇인지 증명해야 한다. SPIFFE 신원, GPU 작업 증명, 디바이스 실링, 수출 통제 KYC 게이트, EU AI 법령 준수 문서 생성 등을 통해 신뢰는 네트워크 위치가 아닌 증거로 확보된다.
- 비용 및 탄소 인식 오케스트레이션 — TCO 모델링, 탄소 인식 배치, 클라우드 버스트, N+K 페일오버 예비 용량, 클라우드 마켓플레이스 어댑터를 통해 가격과 탄소 배출을 스케줄링의 1차 요소로 통합한다.
- 멀티 래프트 합의 — 샤드별 래프트 그룹과 리스홀더 로컬 읽기를 통해 낮은 지연 시간의 일관성을 보장하며, STONITH 펜싱(IPMI/EC2/Azure/SBD)을 통해 장애 노드를 결정적으로 제거하여 상태 오염을 방지한다.
- 드리프트 방지 기계적 린트 — archlint는 문서화된 컴포넌트가 존재하지 않는 패키지 경로에 매핑되는 즉시 빌드를 실패시키고, 문서 체인 엔진은 마크다운/HTML/PDF/DOCX의 바이트 일관성을 유지하여 문서가 코드와 모순되지 않도록 한다.

콘텐츠

가장 큰 기술적 도전 과제와 해결 방법

- **“PASS-bluff”(비기능적 기능에 대한 테스트 통과)**. 프로젝트 전체가 근절하기 위해 설계된 실패 모드: 스텝 기반의 녹색 테스트 스위트. 필수 쌍별 변이 테스트로 해결 — 모든 작업 항목에는 독립적인 코드 변이 시반드시 실패해야 하는 명명된 보호 테스트가 포함되며, 이를 통해 통과한 테스트가 모킹이 아닌 실제 동작을 검증한다는 것을 증명합니다.
- **크로스 플랫폼 동등성(Linux 전용 모킹 금지)**. 빌드 태그를 통해 공유 인터페이스를 실제 OS별 기능으로 분할하여 해결 — Linux cgroup//proc/커널 WireGuard, macOS sysctl/vm_stat/IOKit/wireguard-go — 그리고 독립적인 OS 오라클과 교차 검증을 통해 각 플랫폼이 Linux 가정이 아닌 진정한 네이티브 상태를 보고하도록 합니다.
- **장애 상황에서의 분산 시스템 정확성**. 결정론적 시뮬레이션 테스트와 선형화 가능성 검사기로 해결. 이 검사기는 파티션, 크래시, 클럭 스큐를 인위적으로 생성 및 재생하며, TLA+ 형식 명세에 기반해 코드 한 줄 작성 전에 합의 및 스케줄링 불변성을 명확히 정의합니다.
- **문서화 및 아키텍처 드리프트**. archlint로 해결 — 문서화되었으나 존재하지 않는 매핑 패키지가 발견되면 빌드를 실패시키고, docs_chain 검증

게이트에는 예외 없이 적용됩니다. 드리프트는 오래된 위키 페이지가 아닌 빌드 실패로 처리됩니다.

- 미완료 작업의 정직한 범위 설정. 기밀 다중 노드 추론 왕복은 의도적으로 티켓 뒤에 게이트되어 “아직 엔드투엔드 검증되지 않음”으로 표시되며, 완료된 작업과 동일한 엄격함으로 아직 완료되지 않은 부분에도 적용됩니다.

기술 스택

- **Go (go.mod: 1.25 / toolchain 1.25.4)** — 약 30개 모듈로 구성된 워크스페이스 전체의 제어 플레인 언어로, 저비용 고루틴 동시성과 데이터센터부터 엣지까지 동일한 정적 바이너리 배포를 위해 선택되었습니다.
- **Zig (0.14+) + C/C++** — Go 런타임이 방해가 되는 경우에만 사용: 저수준 시스템 프리미티브와 하드웨어에 대한 결정론적이며 할당 없는 제어가 필요한 GPU 커널.
- **gRPC + Protocol Buffers** — 모든 하위 시스템 간 API(api/v1/)는 타입이 지정된 버전 관리 계약을 따르므로, 14개의 마이크로서비스가 서로를 깨뜨리거나 수동으로 와이어 포맷을 작성하지 않고도 진화할 수 있습니다.
- **Raft (etcd-raft) + SWIM** 가십 — 의도적인 분할: Raft는 반드시 강한 일관성이 요구되는 상태를 담당하며, SWIM 가십은 대규모 멤버십 및 디스커버리에서 합의가 너무 무거울 경우 이를 처리합니다.
- **PostgreSQL 16, Redis 7** 클러스터, **etcd v3.5, SQLite** — 각 작업에 적합한 스토어: 내구성 있는 관계형 상태는 Postgres, 핫 캐시는 Redis, 조정은 etcd, 노드 로컬 HXC 작업 항목 레지스트리는 임베디드 SQLite.
- **NATS 2.10 (JetStream), Kafka 4.0 (KRaft), RabbitMQ 3.13** — 세 가지 메시징 백본으로 세 가지 트래픽 유형 처리: NATS/JetStream은 빠른 내부 이벤팅, Kafka는 내구성 있는 고처리량 스트림, RabbitMQ는 클래식 브로커 의미론.
- **WireGuard** 메쉬 + **ML-KEM-768/X25519 + AES-256-GCM/ChaCha20-Poly1305 + HKDF** — WireGuard로 경량 노드 간 메쉬 구축, 하이브리드 양자 내성 핸드셰이크와 AEAD 레코드로 래핑하여 고전적 및 양자 공격자로부터 전송 계층의 기밀성을 보장합니다.
- **SPIFFE + JWT (HS256) + 범위 기반 RBAC + OPA** — 계층화된 신원 및 권한 부여: SPIFFE는 워크로드 신원, JWT는 토큰, 범위 기반 RBAC는 대략적인 접근 제어, OPA는 세부 정책을 코드로 표현합니다.
- **Prometheus v2.50, Grafana 10.4, Jaeger 1.55, W3C** 트레이싱 — 메트릭, 대시보드, 분산 트레이스와 W3C 컨텍스트 전파로 서비스 및 하드웨어 계층을 넘나드는 요청 추적.
- **HashiCorp Vault 1.16** — 시크릿과 키 자료는 코드와 설정에서 분리되어 감사 하에 발급됩니다.
- **Docker Compose, Kubernetes (kustomize, 강화된 securityContext), Helm** — 로컬 구동을 위한 Compose와 실제 배포를 위한 Kubernetes/Helm(강화된 securityContext), 하나의 정의로 환경 간 프로모션.
- **React + TypeScript + Vite (Node 20+)** — 세션, 터미널, 풀 활용도를 위한 빠르고 타입 안전성이 보장된 웹 UI.

- **TLA+** — 합의 및 스케줄링 불변성에 대한 형식 명세로, 구현 전에 설계 단계에서 가장 테스트하기 어려운 속성을 증명합니다.

내용

상태 및 정직성 안내

- 상태: 개발 중. 현재 버전은 초기 단계(0.1.0-dev)입니다. 완전한 기밀 다중 노드 추론 라운드트립, 마켓플레이스 정산, 증명 기반 스케줄링 인 구 등 여러 고급 기능은 저장소에서 계획됨 / 인프라 제한됨 **(PLANNED / infra-gated)**으로 명시되어 있으며, 완전히 구현된 것으로 제시되지 않습니다. 커버리지 수치는 자가 보고된 것입니다.
- 라이선스: 미정. 명확히 선언되지 않았으며, Helm 차트의 HelixCluster/HelixCluster 및 helixcluster.io URL은 실제 원격 저장소와 일치하지 않는 미검증 플레이스홀더입니다.
- 번들된 LLM 스택 프로젝트(LLMOrchestrator, LLMPProvider, LLMsVerifier)는 클러스터 내 호스팅되는 모델 서버가 아닌 분리된 서브모듈입니다.

우선 순위 계층: Helix-primary(LLM-infrastructure 클러스터 — 추론 및 컴퓨팅 워크로드를 호스팅할 수 있는 컴퓨팅 기반). HelixTrack 다음 순위를 차지합니다.